## MAX HEAP SORT

```
#include <iostream>
using namespace std;

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i)
{
   int largest = i;       // Assume root is largest
   int left = 2*i + 1;    // Left child
   int right = 2*i + 2;   // Right child

   // If left child is larger than root
   if(left < n && arr[left] > arr[largest])
      largest = left;

   // If right child is larger than largest so far
   if(right < n && arr[right] > arr[largest])
      largest = right;

   // If largest is not root
   if(largest != i)
   {
      swap(arr[i], arr[largest]);
      heapify(arr, n, largest);
   }
}

// Heap sort function
```

```
void heapSort(int arr[], int n)
{
    // Build max heap
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for(int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);  // Move current root to end
        heapify(arr, i, 0);    // Heapify reduced heap
    }
}

// Print array
void printArray(int arr[], int n)
{
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    heapSort(arr, n);
```

```
    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

OUTPUT :

```
Output
Sorted array: 5 6 7 11 12 13

=== Code Execution Successful ===
```